

UNIT 1

1. Introduction to Generative Artificial Intelligence

Artificial Intelligence (AI) is a branch of computer science concerned with developing machines and software systems capable of performing tasks that normally require human intelligence.

These tasks include learning from data, reasoning, problem solving, understanding language, recognizing images, making decisions, planning, and understanding speech.

Traditional computer programs generally follow explicitly defined instructions. AI systems, particularly modern machine-learning systems, can instead learn patterns from examples.

For example, instead of programming thousands of rules for identifying cats, a machine-learning model can be trained using many images of cats and non-cats. The model learns patterns such as shapes, textures, edges and spatial relationships and uses these patterns to classify new images.

Generative AI

Generative Artificial Intelligence is a branch of AI that focuses on developing models capable of creating new data or content that resembles the data on which the model was trained.

The generated content may include text, images, audio, music, video, computer programs, 3D objects, synthetic datasets, and designs.

A traditional image classifier may identify an image as a cat, whereas a Generative AI model may create a new image of a cat. A traditional text classifier may identify a sentence as positive or negative, whereas a Generative AI model may write a new paragraph.

Thus, Generative AI is concerned not only with understanding existing data, but also with creating new data.

Basic Principle of Generative AI

The fundamental principle behind Generative AI is learning the underlying patterns and probability distribution of the training data.

If real-world data is represented by a probability distribution $p_{\text{data}}(x)$, a generative model attempts to learn an approximation $p_{\text{model}}(x) \approx p_{\text{data}}(x)$. Once the model has learned the distribution sufficiently well, it can generate new samples from the learned distribution.

For example, after learning from thousands of photographs of human faces, a model can learn characteristics such as facial shape, eye position, nose shape, hair patterns, skin texture, lighting, and relationships between facial features. It can then generate a new face that was not directly present in the training dataset.

2. Generative AI vs Traditional AI

Traditional or discriminative AI systems are often designed to analyze, classify, predict, or make decisions. Generative AI systems are designed to produce new content.

Traditional AI generally learns decision boundaries or mappings from inputs to outputs. Generative AI attempts to model patterns or distributions that enable it to produce new samples.

Examples include spam detection versus email generation, image classification versus image generation, and sentiment analysis versus text generation.

Evolution of Generative AI

Generative AI developed through several stages of AI and machine-learning research. Early rule based AI relied on manually written rules. Statistical machine learning introduced learning from data through algorithms such as linear regression, logistic regression, decision trees, Naive Bayes, and Support Vector Machines. Neural networks introduced layered computational representations, and the deep-learning revolution was accelerated by large datasets, powerful GPUs, and improved optimization and network architectures.

Major generative model families include Generative Adversarial Networks, Variational Autoencoders, autoregressive models, Transformers, and diffusion models.

Applications of Generative AI

Generative AI is used for text generation, code generation, image generation, audio and music generation, video generation, synthetic data generation, design, and many other creative and technical tasks.

In text generation it can produce articles, stories, emails, reports, summaries and explanations. In code generation it can produce programs, SQL queries, scripts, documentation and test cases. In synthetic data generation it can create artificial datasets for testing, simulation, training, and privacy sensitive applications.

3. Generative Adversarial Networks (GANs)

Definition

A Generative Adversarial Network (GAN) is a generative deep-learning architecture consisting of two neural networks called the Generator and the Discriminator. These networks are trained simultaneously in an adversarial manner.

The Generator attempts to create realistic fake samples, while the Discriminator attempts to distinguish between real and generated samples.

Basic GAN Architecture

The Generator receives a random latent vector or noise vector and transforms it into a synthetic sample. The Discriminator receives both real training samples and generated samples and produces a score indicating whether each sample is real or fake.

Conceptually:

Random Noise → Generator → Fake Sample → Discriminator → Real/Fake

Real Data → Discriminator → Real/Fake

Generator

The Generator is responsible for producing synthetic samples. If z represents a random noise vector sampled from a prior distribution $p(z)$, the Generator transforms it into $G(z)$. Initially the output may look like random noise. Through adversarial training, the Generator gradually learns to produce samples that resemble the real training distribution.

Discriminator

The Discriminator distinguishes real samples from generated samples. It can be represented as $D(x)$, where the output is interpreted as the probability that x is real. A value close to 1 indicates a real prediction, while a value close to 0 indicates a fake prediction.

Working of GAN

GAN training proceeds through repeated competition. First, real samples are selected. Random noise is sampled. The Generator creates fake samples. The Discriminator is trained using both real and fake samples. The Generator is then updated so that its generated samples are more likely to be classified as real. This cycle is repeated over many iterations.

Ideally, the Generator improves its ability to model the real distribution while the Discriminator improves its ability to distinguish real and generated samples.

GAN Objective Function

The original GAN objective is expressed as:

$$\min_G \max_D V(D, G)$$

where:

$$V(D, G) = E[x \sim p_{\text{data}}(x)] [\log D(x)] + E[z \sim p_z(z)] [\log(1 - D(G(z)))]$$

The first term concerns real data and the second term concerns generated data. The Discriminator attempts to maximize the objective while the Generator attempts to minimize it in the original minimax formulation.

GAN Equilibrium

The theoretical objective of GAN training is for the generated distribution $p_g(x)$ to approach the real distribution $p_{\text{data}}(x)$. At the ideal equilibrium, $p_g(x) = p_{\text{data}}(x)$. The Discriminator then cannot reliably distinguish real samples from generated samples, giving $D(x)$ approximately 0.5 for both distributions under the idealized theoretical setting.

Advantages and Limitations

Advantages include high-quality generation, no need for explicit density estimation in the original framework, strong performance in image synthesis, architectural flexibility, and synthetic-data generation.

Limitations include training instability, mode collapse, sensitivity to hyperparameters, difficulty in evaluating generation quality and diversity, and the need to balance Generator and Discriminator learning.

Mode Collapse

Mode collapse occurs when the Generator produces a limited variety of samples. For example, if the training dataset contains many different types of faces but the Generator repeatedly produces very similar faces, the outputs may look realistic but fail to represent the diversity of the original distribution.

Possible causes include unstable adversarial optimization, exploitation of weaknesses in the current Discriminator, poor balance between Generator and Discriminator, and limited exploration of the data distribution.

GAN Variants

DCGAN (Deep Convolutional GAN) uses convolutional neural networks for image generation.

Conditional GANs generate samples according to an additional condition y , represented as $G(z, y)$.

Wasserstein GAN modifies the objective using Wasserstein distance to improve training stability and gradient behavior. CycleGAN performs image-to-image translation without requiring paired training examples and uses cycle consistency to preserve important content.

4. Encoder–Decoder Architecture

Definition

An Encoder–Decoder architecture consists of two major components: an Encoder and a Decoder. The Encoder converts an input into an internal representation, and the Decoder uses that representation to produce the output.

General flow:

Input → Encoder → Representation → Decoder → Output

Encoder

The Encoder processes input data and extracts meaningful information. In language applications, it converts a sequence into numerical representations capturing information such as semantic meaning, syntax, context, and relationships between words.

Decoder

The Decoder uses the encoded information to produce the desired output. Encoder–Decoder architectures are particularly useful for translation, summarization, question answering, caption generation, and other sequence transformation tasks.

5. Sequence-to-Sequence Models

Definition

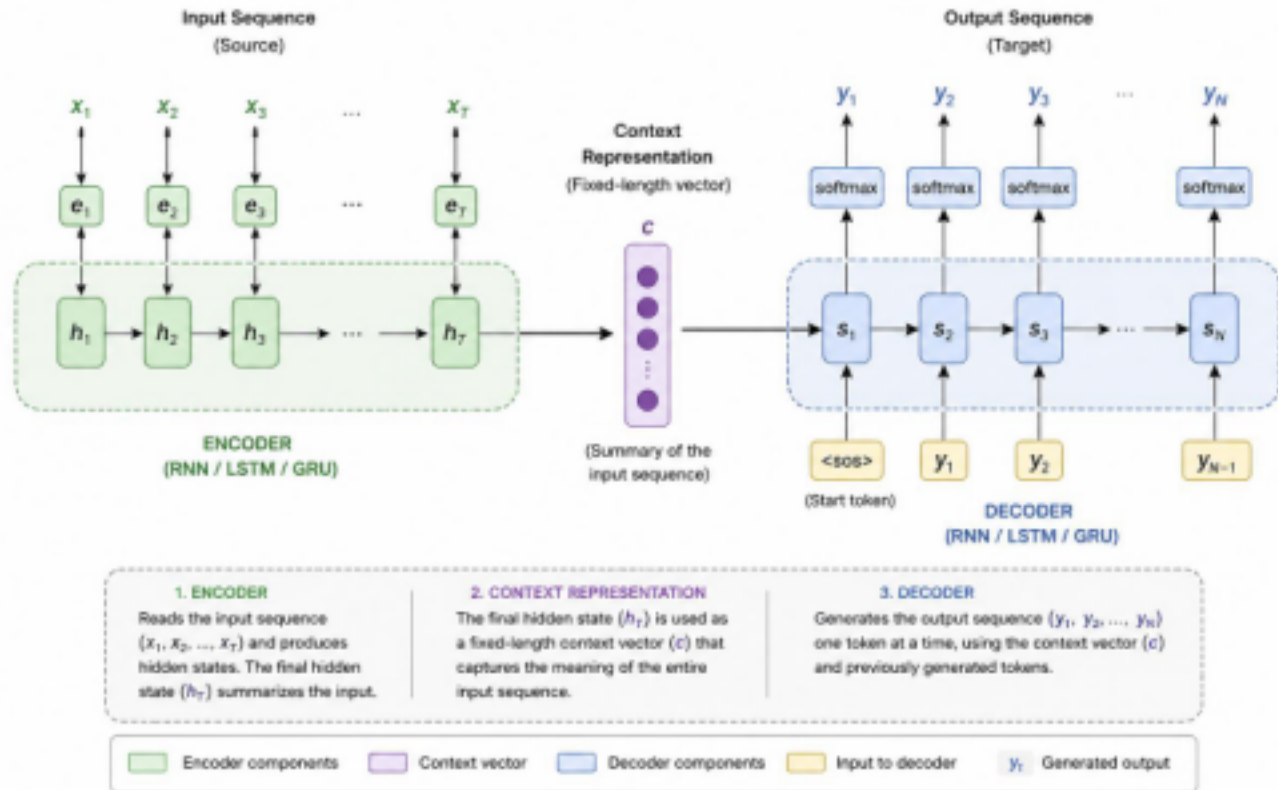
A Sequence-to-Sequence (Seq2Seq) model transforms one sequence into another sequence. The input and output sequences may have different lengths. For example, an English sentence can be converted into a French sentence.

Traditional Seq2Seq systems commonly used recurrent neural networks such as RNNs, LSTMs, or GRUs.

Traditional Seq2Seq System

A traditional Seq2Seq system consists of an Encoder, a context representation, and a Decoder. The Encoder reads the input sequence and creates a representation. The Decoder uses this representation to generate the output sequence.

Traditional Seq2Seq System



Limitation

A major limitation of early Encoder–Decoder systems was the fixed-size context representation. Compressing a long sentence into a single vector can cause information loss. The attention mechanism addressed this by allowing the Decoder to focus on different parts of the input at different generation steps.

6. Variational Autoencoders (VAEs)

Definition

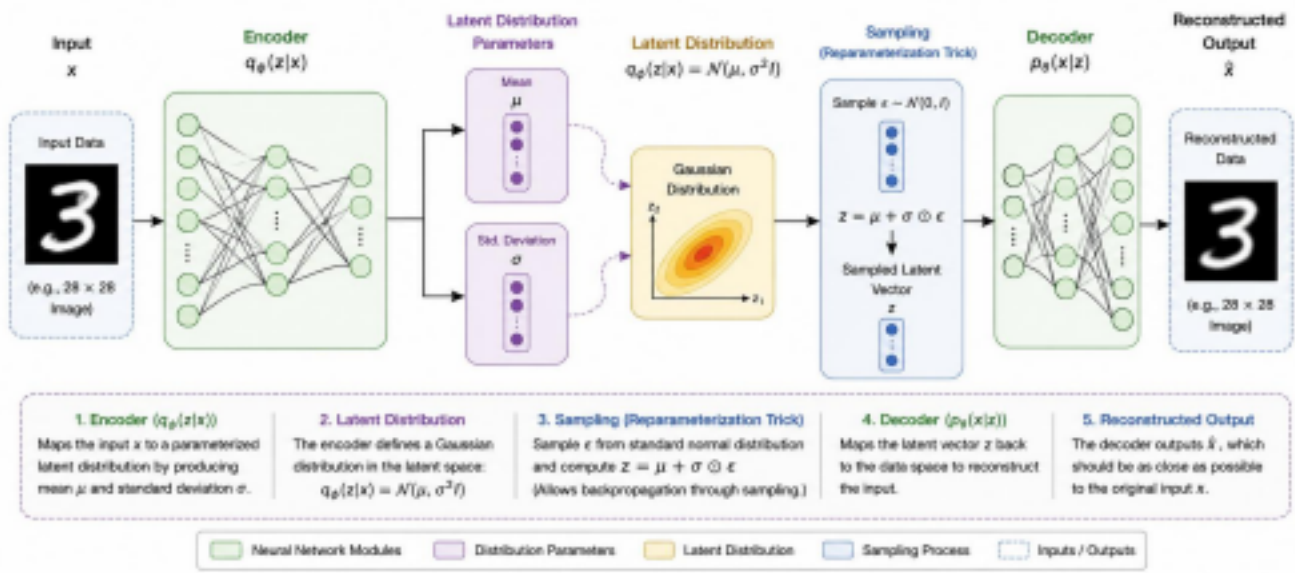
A Variational Autoencoder (VAE) is a probabilistic generative model that learns a structured latent representation of data. It combines an Encoder, a latent probability distribution, sampling, and a Decoder.

Architecture

The VAE Encoder produces parameters of a probability distribution, typically a mean μ and a standard deviation σ . A latent variable z is sampled from this distribution and passed to the Decoder, which reconstructs the input.

Input $x \rightarrow$ Encoder $\rightarrow \mu, \sigma \rightarrow$ Latent distribution $\rightarrow$ Sampling $\rightarrow z \rightarrow$ Decoder $\rightarrow$ Reconstructed x

Variational Autoencoder (VAE) Architecture



Reparameterization Trick

Random sampling can make direct backpropagation difficult. The reparameterization trick expresses the latent variable as:

$$z = \mu + \sigma \odot \epsilon$$

where ϵ is sampled from a standard normal distribution $\mathcal{N}(0, I)$. This separates the randomness from the learnable parameters and allows gradients to flow through μ and σ .

VAE Loss Function

The VAE loss contains two major components:

$$L = L_{\text{reconstruction}} + L_{\text{KL}}$$

Reconstruction loss measures how accurately the Decoder reconstructs the original input. KL divergence encourages the learned latent distribution to remain close to a chosen prior, usually $\mathcal{N}(0, I)$. The combination encourages good reconstruction and a smooth, useful latent space.

Advantages and Limitations

VAEs provide structured latent spaces, probabilistic generation, smooth interpolation, relatively stable training, and useful representation learning. Applications include image generation, anomaly detection, representation learning, data generation, and dimensionality reduction.

A major limitation of basic VAEs is that generated images may appear blurry compared with high quality GAN outputs, partly because probabilistic reconstruction objectives can average over multiple plausible outputs.

7. Transformers

Definition

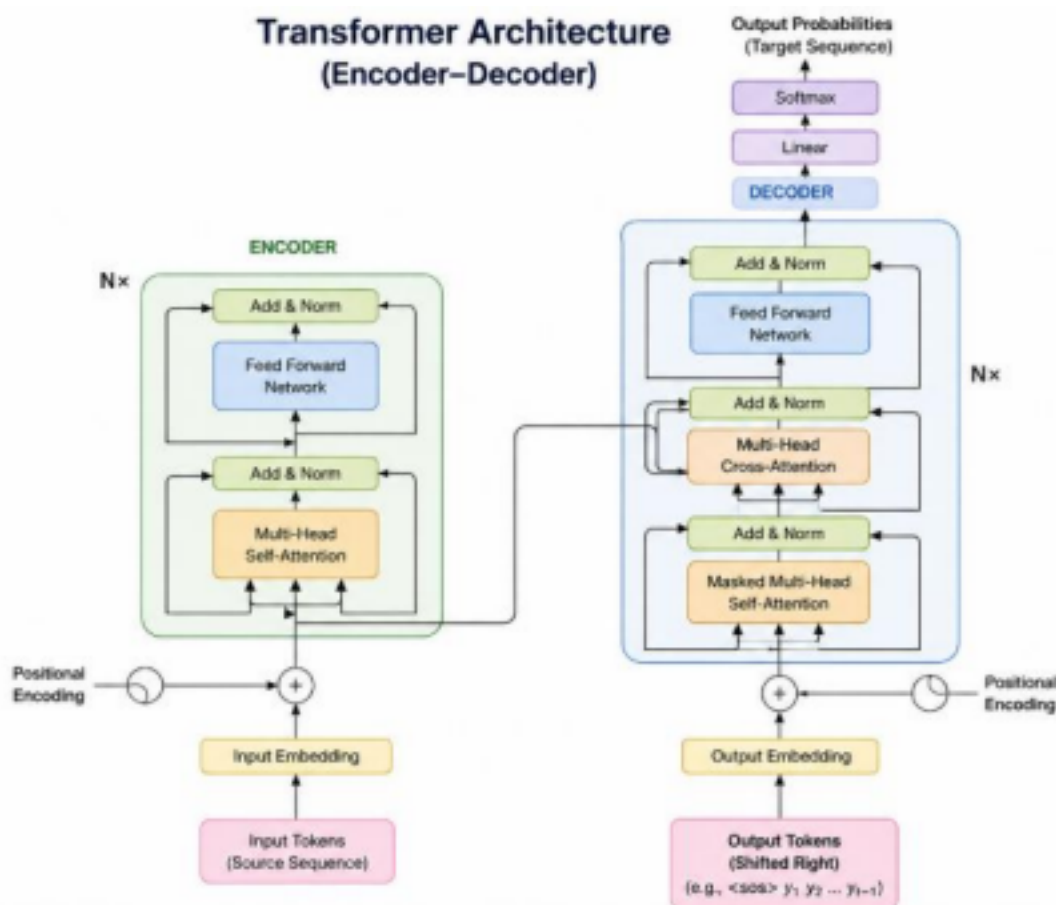
A Transformer is a neural-network architecture based primarily on attention mechanisms. It became highly important for Natural Language Processing, Generative AI, and large language models.

Transformers address limitations associated with recurrent sequence processing by enabling highly

parallelizable computation during training and direct attention-based interactions among tokens.

Architecture

A standard Transformer contains Encoder and Decoder stacks. Encoder layers contain multi-head self-attention, feed-forward networks, residual connections and normalization. Decoder layers contain masked self-attention, cross-attention, feed-forward networks, residual connections and normalization.



Self-Attention

Self-attention allows each token to examine other tokens in the same sequence. This enables the model to capture relationships between distant words and construct contextual representations.

For example, in a sentence containing a pronoun such as 'it', attention can help the model examine other words to determine which entity is relevant.

Query, Key and Value

Self-attention uses Query (Q), Key (K), and Value (V) representations. The Query represents what the current token is looking for, the Key represents information used for matching, and the Value contains information that is combined.

The scaled dot-product attention equation is:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Multi-Head Attention

Instead of performing attention once, Transformers perform attention through multiple heads. Each

head can learn different relationships. The outputs of the heads are concatenated and passed through a linear transformation. This increases the representational capacity of the model.

Positional Encoding

Self-attention does not inherently specify the order of tokens. Positional information is therefore incorporated so that the model can distinguish different token orders. Approaches include sinusoidal positional encoding and learned positional embeddings, along with other modern positional mechanisms.

Feed-Forward Network

After attention, each token representation is passed through a feed-forward neural network. A simplified expression is:

$$\text{FFN}(x) = W_2 f(W_1 x + b_1) + b_2$$

The feed-forward component provides nonlinear transformation of token representations.

Residual Connections

Transformers use residual connections of the form $\text{Output} = x + F(x)$. These help preserve information, improve gradient propagation, and make deep networks easier to optimize.

8. BERT

Definition

BERT stands for Bidirectional Encoder Representations from Transformers. It is a Transformer-based language model designed primarily for language understanding. BERT uses the Encoder portion of the Transformer architecture.

Bidirectional Context

BERT constructs contextual representations using information from both left and right sides of a token. This allows the meaning of a word to be interpreted using its surrounding context.

Pretraining

One important original BERT pretraining objective is Masked Language Modeling. Some tokens are masked and the model predicts the missing tokens using surrounding context. The original BERT formulation also included Next Sentence Prediction, although later research explored alternative pretraining objectives.

Example:

The cat is [MASK] on the mat.

The model predicts a likely missing word such as 'sleeping'.

Applications

BERT can be used for sentiment analysis, text classification, named entity recognition, question answering, semantic similarity, and information extraction. It is primarily associated with language understanding rather than open-ended text generation.

9. GPT

Definition

GPT stands for Generative Pre-trained Transformer. GPT models are Transformer-based language models that use an autoregressive, decoder-style architecture. The fundamental task is to predict the next token based on previously available tokens.

Generation Process

The generation process can be summarized as:

Prompt → Tokenization → Token embeddings → Transformer blocks → Next-token probabilities → Select/sample token → Append token → Repeat.

The process continues until an end condition or maximum length is reached.

Autoregressive Generation

GPT models generate text autoregressively. The probability of a complete sequence can be decomposed as:

$P(x_1, x_2, \dots, x_n) = \text{product from } t=1 \text{ to } n \text{ of } P(x_t \mid x_1, \dots, x_{(t-1)})$

This means that each next-token probability is conditioned on previously generated tokens.

10. T5

Definition

T5 stands for Text-to-Text Transfer Transformer. It is an Encoder–Decoder Transformer architecture that treats many NLP tasks as text-to-text problems.

The central idea is that every task is represented as text input and text output.

Examples

For translation:

Input: translate English to French: Hello

Output: Bonjour

For summarization:

Input: summarize: [long article]

Output: [summary]

For classification:

Input: sentiment: The movie was excellent.

Output: positive

This unified formulation allows a common text generation framework to be applied to multiple tasks.

11. BERT vs GPT vs T5

Comparison

BERT uses an Encoder-only architecture and is mainly designed for language understanding with bidirectional context. GPT uses a Decoder-only architecture and is designed for autoregressive generation. T5 uses an Encoder–Decoder architecture and treats tasks as text-to-text

transformations.

Easy way to remember:

BERT → Understand

GPT → Generate

T5 → Transform

12. Transformer vs RNN

Comparison

RNNs process sequences sequentially and maintain recurrent hidden states. Transformers use attention and can process many sequence positions in parallel during training. Transformers are better suited to capturing long-range relationships and large-scale parallel training, which has contributed to their dominance in modern NLP and Generative AI.

13. Sampling and Decoding

Greedy Decoding

Greedy decoding selects the highest-probability token at every step. It is simple and deterministic but can sometimes produce repetitive or overly predictable text.

Temperature

Temperature controls the randomness of token selection. Lower temperature produces more predictable output, while higher temperature produces more diverse output. Temperature changes the sharpness of the probability distribution used during sampling.

Top-k Sampling

Top-k sampling restricts token selection to the k most probable candidates. For $k = 5$, only the five highest-probability tokens are considered. This reduces the chance of sampling extremely unlikely tokens.

Top-p / Nucleus Sampling

Top-p sampling selects the smallest set of candidate tokens whose cumulative probability reaches a chosen threshold. For example, with $p = 0.9$, the model considers the smallest group whose combined probability is approximately 90%. The number of candidates can therefore change depending on model uncertainty.

14. Transfer Learning and LLMs

Transfer Learning

Transfer learning refers to using knowledge learned from one task or dataset for another related task. A typical workflow is large-scale pretraining, learning general representations, followed by fine-tuning or adaptation for a specific task.

A pretrained language model can be adapted for customer support, medical text analysis, legal documents, code generation, and many other applications.

Large Language Models

A Large Language Model (LLM) is a language model trained with very large amounts of text and usually a large number of parameters. Modern LLMs commonly use Transformer architectures. They can learn grammar, vocabulary, syntax, semantic relationships, contextual associations, and programming patterns.

15. Advantages, Limitations and Ethics

Advantages

Transformer-based Generative AI provides parallelizable training, strong handling of long-range dependencies, transfer learning, scalability, and potential for multimodal applications.

Limitations

Important limitations include hallucination, bias, privacy concerns, computational cost, and evaluation difficulty. A model can produce fluent but incorrect information, so fluency should not be treated as proof of factual accuracy.

Ethical Issues

Important ethical concerns include bias, misinformation, deepfakes, copyright and intellectual-property questions, privacy, and the need for appropriate human oversight in high-impact applications.

16. Important Comparisons

GAN vs VAE vs Transformer

GANs use adversarial learning with a Generator and Discriminator and are particularly associated with realistic synthesis. VAEs use probabilistic latent representations and a reconstruction plus KL divergence objective. Transformers use attention and are especially powerful for sequence modeling, language, and large-scale Generative AI.

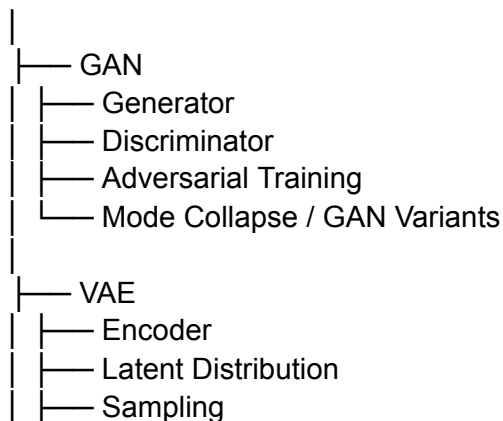
GAN: Generator + Discriminator

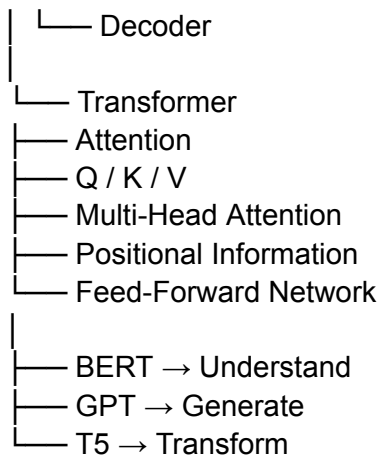
VAE: Encoder + Latent Distribution + Decoder

Transformer: Attention + Feed-Forward Networks

Conceptual map

Generative AI





Key points:

GAN: Generator creates → Discriminator evaluates → adversarial feedback → Generator improves. VAE: Encode → learn distribution → sample latent vector → decode → reconstruct/generate. Seq2Seq: Input sequence → Encoder → representation → Decoder → output sequence. Transformer: Embeddings → positional information → attention → feed-forward network → output. BERT: Encoder-only + bidirectional context + language understanding. GPT: Decoder-only + autoregressive next-token prediction + generation. T5: Encoder–Decoder + every task formulated as text → text.